

基于子环 Ring Switching 的 CKKS Bootstrapping 分解

初代成果汇总版本

梁朝阳

2026 年 4 月 30 日

摘要

CKKS bootstrapping 的主要开销来自线性变换和模约减核心。本文研究一个大环 CKKS bootstrapping 是否能够被分解为若干个小环上的 leaf bootstrapping。给定 $N = kn$ ，我们将 HERMES-style 子环 ring switching 与明文多项式的 coefficient/module decomposition 结合起来。本文的主要理论主张是：传统 CKKS bootstrapping core

$$S2C_N \circ \text{EvalMod}_N \circ C2S_N$$

与该分解兼容。换言之，split 之后可以在每个 leaf 上独立执行

$$S2C_n \circ \text{EvalMod}_n \circ C2S_n,$$

然后再 merge 回大环。该构造不需要在 split 之后立即恢复原始的大环 slots。本文还讨论 coefficient/slot 双视角下的 split 语义、一个更一般的两步骤拆分解、实现诊断、参数选择，以及 reduced-modulus leaf bootstrapping 这一开放问题。

[待完成：在安全估计、key-size 表格和 reduced- Q 实验完成之后，替换为最终摘要。]

目录

1 引言	2
1.1 研究动机	2
1.2 核心观察	2
1.3 本文贡献	3
1.4 与已有工作的关系	3
2 Preliminaries	4
2.1 CKKS ciphertext 语义与核心参数	4
2.2 CKKS bootstrapping 的 coefficient view 与 slot view	5
2.3 同态实现中常用的基础原语	6
2.4 Ring switching 与 coefficient/module decomposition	7
2.5 Coefficient split 与 slot-preserving split 的最小区别	8
2.6 RLWE 安全性与参数选择	8

3	子环分解的 Coefficient/Slot 语义	9
3.1	密文表示语义：我们实际操作的是 <code>ct</code>	9
3.2	Coefficient/module split：密文下的目标语义	9
3.3	子环嵌入与压缩的 slot 几何	11
3.4	Coefficient split 在 slot 视角下发生什么	12
3.5	Trace-like projection 的补充视角	13
3.6	Slot split 与 slot-preserving split	13
3.7	Slot-level split as representation switching	14
3.8	Two-step splitting interpretation	15
3.9	为什么 bootstrapping 分解选择 coefficient split	15
3.10	本节小结	16
3.11	本节小结	16
4	主要理论结果	16
4.1	Notations	17
4.2	引理与定理陈述	17
4.3	Split-C2S compatibility 的证明	18
4.4	EvalMod compatibility 的证明	19
4.5	S2C-Merge compatibility 的证明	20
4.6	CKKS core factorization 的证明	21
4.7	General split-compatible sandwich theorem	21
4.8	本节小结	23
5	算法、正确性与参数分析	23
5.1	Factored bootstrapping algorithm	23
5.2	正确性与噪声	23
5.3	复杂度与 overhead	24
5.4	安全性与参数选择	24
5.5	安全证明	25
6	实现与实验评估	25
6.1	Implementation setup	25
6.2	Layout and normalization diagnostics	25
6.3	Correctness experiments	25
6.4	Performance and overhead	25
6.5	RNS-aware reduced- Q investigation	26
7	讨论与限制	26
8	结论	26

1 引言

前面可能需要写点大同行看的

1.1 研究动机

CKKS bootstrapping 通过同态方式计算一个类似解密的电路，从而刷新密文。在传统视角下, ModRaise 或 ModUp 之后, 明文多项式中会出现形如 $m + q_0 I$ 的项。随后 bootstrapping core 将 coefficient 信息搬到 slots 中, 逐坐标执行近似 modular reduction, 再将结果搬回 coefficient 表示。我们将该核心写成

$$\text{BTS}_N^{\text{core}} = \text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N.$$

其中昂贵的不只是 EvalMod 中的标量近似, 也包括大环上的线性变换 C2S_N 和 S2C_N。因此, 本文的核心问题是: 当 $N = kn$ 时, 能否将 $R_{q,N}$ 上的一次 bootstrapping 分解为 $R_{q,n}$ 上的若干次更小的 bootstrapping?

本文的答案是: 利用 subring ring switching, 将一个大环 ciphertext 转换为 k 个 leaf ciphertexts, 在每个 leaf 上执行 bootstrapping core, 然后合并输出。整体形态为

$$\text{RSUp}_{n \rightarrow N} \circ \left(\bigoplus_{r=0}^{k-1} \text{BTS}_n \right) \circ \text{RSDown}_{N \rightarrow n}.$$

这里的 RSDown 和 RSUp 是 ciphertext-level ring switching 操作, 包含 key switching, 并不是单纯的 coefficient rearrangement。代数层面的 coefficient decomposition 很便宜; 而密文层面的转换需要 switching keys, 并会带来额外噪声。

1.2 核心观察

设 $P(X) \in R_N$ 且 $N = kn$ 。coefficient/module decomposition 将其写成

$$P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k).$$

关键观察不是这个 split 保留了大环 slots; 事实上它并不保留。真正重要的是: 这个 coefficient split 与 $\text{C2S} \rightarrow \text{EvalMod} \rightarrow \text{S2C}$ 的结构兼容。更具体地说, split 之后执行 leaf C2S_n, 等价于先执行 top-level C2S_N, 再按照 coefficient residue classes 对坐标分组, 差别只在一个 layout permutation。由于 EvalMod 是逐坐标作用, 它可以在这些分组上独立执行。最后, leaf S2C_n 加上 merge 可以恢复 top-level S2C_N 的语义。

因此, 大环 core 可以替换为以下 leaf-wise core:

$$\text{Split}_k \rightarrow \bigoplus_r \text{C2S}_n \rightarrow \bigoplus_r \text{EvalMod}_n \rightarrow \bigoplus_r \text{S2C}_n \rightarrow \text{Merge}_k.$$

如果从 slot-preserving split 的角度出发, 某种 slot-lifting 或 slot-recovery transform 看起来会自然出现。本文的构造不将其作为算法步骤, 而是直接在 coefficient/module layout 下工作。

1.3 本文贡献

本文计划形成三个贡献。

贡献一：CKKS bootstrapping 的 coefficient-split factorization。 我们证明传统 CKKS bootstrapping core 可以在不需要额外线性变换的情况下通过 coefficient/module decomposition 分解为 leaf bootstrapping core。更具体地，对于 $N = kn$ ，有

$$\text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N = \text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{S2C}_n \circ \text{EvalMod}_n \circ \text{C2S}_n \right) \circ \text{Split}_k,$$

其中需要考虑 implementation layout conventions 和 approximation error。这个贡献合并了 subring-factored bootstrapping construction 和 split-compatible core theorem。它同时说明：split 之后不需要先恢复原始 big-ring slots。

贡献二：coefficient/slot 双视角和两步骤拆分解释。 我们详细解释 coefficient split 与 slot-preserving split 的区别。对于 $k = 2$ ，coefficient split $P(X) = A(X^2) + XB(X^2)$ 将一对 big-ring slot values $P(\alpha), P(-\alpha)$ 映射为局部线性组合 $A(\theta)$ 和 $B(\theta)$ ，而不是直接保留原始 slot values。我们还描述一个更一般的 two-step interpretation：第一步执行 coefficient/module decomposition；第二步，如果应用确实需要某种特定 slot 语义，则再执行额外的 coordinate transform。这个 two-step view 更一般，也有理论意义，但因为会引入额外 linear-transform cost，它不是本文的主算法路径。

贡献三：实现、参数分析与 reduced-modulus 探索。 我们在 Lattigo 上实现并诊断该构造，包括 layout alignment、implementation normalization、direct-vs-factored error、output level、timing 和 security-aware 参数选择。我们还分析 reduced- Q 问题：ring switching 之后，leaf bootstrapping 是否可以使用更短的 modulus chain。当前证据表明，在 coefficient/RNS 层面存在代数调度自由，但现成 Lattigo preprocessing 不能通过简单重排直接下放到 leaves。因此，reduced- Q 应被视为一个新的 leaf-parameter design problem，而不是 direct path 的中间态等价问题。

[待完成：在最终确定 reduced- Q study 能否作为主文实验之后，精炼三条贡献。]

1.4 与已有工作的关系

这里成正文的时候需要梳理一下行文。

传统 ring switching。 Ring switching 将大环 ciphertext 转换为一个或多个小环 ciphertext。在 packed setting 中，已有工作通常需要在代数分解之后执行额外的 linear transformation，以恢复或重新分配原始 slot values。本文的不同点是：我们不在 split 后立即恢复 big-ring slots。相反，我们证明 leaf C2S 自然产生 bootstrapping core 所需的 coordinate grouping。

HERMES。 HERMES 是 ring packing 和 transciphering 框架，目标是把许多 LWE-format ciphertexts packing 成一个 RLWE/CKKS ciphertext。它通过 low-modulus base ring packing、ring switching、HalfBTS 和 MLWE 技术降低时间和 key material。本文的输入输出不同：输入本来就是一个 CKKS/RLWE ciphertext，目标是刷新它自身。我们使用 HERMES-style ring switching 作为工具，但不声称 ring switching 本身是本文的新贡献。

Subring secret encapsulation。 Subring secret encapsulation 在 bootstrapping 前将 secret 切换到 subring dense secret。它的收益来自更小的 subring-secret Hamming weight、降低 EvalMod 或 digit-removal 成本，以及在 C2S/S2C 中使用 hoisted key switching。本文方法则是真正将 ciphertext 分解为 leaf ciphertexts，并在每个 leaf 上执行 bootstrapping core。简言之，subring secret encapsulation 是 subring-secret acceleration；本文是 full CKKS bootstrapping 的 subring-ciphertext decomposition。

PaCo 及相关新 bootstrapping 设计。 PaCo 通过 partial CoeffToSlot、blind rotations、circle-group modular additions 和 alternative ring structures 重新设计 CKKS bootstrapping。本文不重写 CKKS decryption equation，也不替代传统 EvalMod 路径。本文保留传统的 C2S $\rightarrow$ EvalMod $\rightarrow$ S2C core，只改变该 core 的执行粒度。

传统 CKKS bootstrapping 优化。 许多工作优化传统 bootstrapping pipeline 内部的 linear transforms、EvalMod approximation、precision 或参数选择。本文方法与这些工作正交：我们不是只优化某一个矩阵或近似多项式，而是将整个 core factor 成多个更小的 cores。

[待完成：补完整 bibliographic citations，并加入一张简洁的 related-work comparison table。]

2 Preliminaries

本节只介绍后文需要的 CKKS bootstrapping 背景、若干实现原语、ring switching 抽象和安全参数。我们不在这里完整介绍 CKKS 方案，也不详细展开 coefficient/slot split 的语义；后者将在 Section 3 中专门讨论。本文主要关注的问题是：给定一个需要 bootstrapping 的 CKKS 密文，能否通过子环 ring switching 将其 bootstrapping core 分解为多个 leaf bootstrapping cores。

2.1 CKKS ciphertext 语义与核心参数

令

$$R_{q,N} = \mathbb{Z}_q[X]/(X^N + 1),$$

其中 N 通常为二的幂。一个二元 CKKS ciphertext 写作 $\mathbf{ct} = (c_0, c_1) \in R_{q,N}^2$ 。在 secret $s \in R_{q,N}$ 下，它满足形如

$$\text{Dec}_s(\mathbf{ct}) = c_0 + c_1 s = m + e \pmod{q}$$

的解密关系，其中 m 是经过 CKKS 编码后的 plaintext polynomial， e 是小误差。不同实现可能采用 $c_0 - c_1 s$ 或 $c_0 + c_1 s$ 的符号约定；本文统一使用 $c_0 + c_1 s$ 。如果某个密文在 scale Δ 下表

示消息多项式 $P(X)$ ，我们有时写作 $\mathbf{ct} \models_{\Delta, N} P(X)$ 。这个记号只表达密文的消息语义，不表示 $P(X)$ 可见。

CKKS 同时维护 modulus chain、scale 和 level。通常 modulus chain 以 RNS primes 的乘积表示：

$$Q = \prod_i q_i.$$

每次乘法和 rescale 会消耗 modulus chain 中的一部分 primes。scale Δ 决定近似编码精度；常见实现会选择 2^{35} 、 2^{40} 或类似大小的 scale。bootstrapping 需要额外的 level 来执行 C2S、EvalMod、S2C 以及内部的 rotations、key switching 和 rescale。

除了 ciphertext modulus Q ，CKKS 的 key switching、rotation keys、relinearization keys、bootstrapping keys 和本文的 ring switching keys 通常还使用 auxiliary modulus P 。因此，在安全分析中不能只看输出 ciphertext 当前剩余的 Q ，还需要考虑公开 evaluation keys 中可能出现的最大模数。本文用

$$q_{\max} \approx QP$$

作为保守的工程估计。更精确的安全声明应对 top ciphertext、leaf bootstrapping keys、split/merge switching keys 分别跑 lattice estimator。

注 1 (参数 N, Q, P 的角色). N 同时是容量参数、安全参数和性能成本参数。增大 N 可以容纳更多 slots，也可以承受更大的 Q 或 QP ，但会增加 NTT、rotation、key switching、密钥和内存成本。增大 Q 或 P 可以支持更深的计算或更精确的 key switching，但会降低 RLWE 安全性，因此通常需要更大的 N 。

[待完成：最终正文需要固定所有实现符号： N 是 ring degree；full complex slots 为 $N/2$ ；Lattigo 中的 LogSlots、half-slot convention 和本文 full embedding 记号的对应关系需要在实现章节说明。]

2.2 CKKS bootstrapping 的 coefficient view 与 slot view

CKKS bootstrapping 的目标是刷新一个 modulus 已经接近耗尽的 ciphertext。设输入 ciphertext 在低模数 q_0 下表示 $m(X)$ 。ModRaise 或 ModUp 将它重新提升到更大的 modulus chain 中，但这一步会把消息关系变成类似

$$m(X) + q_0 I(X)$$

的形式，其中 $I(X)$ 是某个 integer polynomial。bootstrapping 的任务是同态去掉 $q_0 I(X)$ ，从而得到新的、具有较高 level 的 $m(X)$ 的加密。

从 coefficient view 看，bootstrapping 处理的是 plaintext polynomial 的 coefficient-like 信息。粗略地说，C2S 将这些待处理的 coefficient values 转入 slots；EvalMod 在 slots 中对每个坐标执行近似 modular reduction；S2C 再把结果带回 coefficient representation。于是标准 core 可以写作

$$\text{BTS}_N^{\text{core}} = \text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N.$$

本文的理论部分主要讨论这个 core，而不把 ScaleDown、ModUp、rescale schedule 和实现 normalization 混入主定理。

从 slot view 看, **EvalMod** 是逐坐标函数: 它在每个 slot 上独立近似 $x \mapsto x \bmod q_0$ 或相应的 scaled modular reduction。正是这个逐坐标性, 使得只要 C2S 后的坐标能够按 residue class 分组, **EvalMod** 就可以在 leaf 上独立执行。本文主定理的核心并不是重新设计 **EvalMod**, 而是证明

$$\text{Split}_k \longrightarrow \bigoplus_r \text{C2S}_n \longrightarrow \bigoplus_r \text{EvalMod}_n \longrightarrow \bigoplus_r \text{S2C}_n \longrightarrow \text{Merge}_k$$

在理想消息语义下等价于 top-level 的 $\text{C2S}_N \rightarrow \text{EvalMod}_N \rightarrow \text{S2C}_N$ 。

需要注意, 本文的 C2S 和 S2C 是 logical bootstrapping transforms。实际实现中, 它们会被分解为许多 baby-step/giant-step rotations、diagonal linear transforms、rescale 和 key switching; 还可能涉及 real/imag branches 或 two-ciphertext layout。这些实现细节会影响常数、level、normalization 和 key size, 但不改变本节中使用的抽象结构。

注 2 (为什么强调两种视角). Coefficient split 在系数层面最自然; **EvalMod** 在 slot 层面最自然。本文的证明正是连接这两个视角: split 后接 leaf C2S_n 并不恢复原始 big-ring slots, 而是进入与 top C2S_N 输出一致的 coefficient-packed 子坐标。

2.3 同态实现中常用的基础原语

后文会抽象使用 linear transform、ring switching 和 bootstrapping evaluator。这里先列出几个常见同态原语, 以便读者理解实现层面的成本来源。

Automorphism. 在 negacyclic ring $R_{q,N} = \mathbb{Z}_q[X]/(X^N + 1)$ 中, automorphism 通常由奇数 $a \in (\mathbb{Z}/2N\mathbb{Z})^*$ 指定:

$$\text{Aut}_a : X \mapsto X^a.$$

在 CKKS 中, automorphism 实现 slot rotation、conjugation 或某些结构化变换, 但密文执行 Aut_a 后通常还需要 key switching, 将 secret 从 $s(X^a)$ 切回目标 secret $s(X)$ 。例如在 $R_{q,2n}$ 中, Aut_{1+2n} 对应 $X \mapsto X^{1+2n} = -X$ 。这个操作可以用于构造 even/odd projection 的同态版本, 例如 $F(X)$ 与 $F(-X)$ 的组合。

Plaintext multiplication. $\text{MulPt}_g(\text{ct})$ 表示用明文多项式 $g(X)$ 乘密文所表示的消息。若 $\text{ct} \models P(X)$, 则理想语义下 $\text{MulPt}_g(\text{ct}) \models g(X)P(X)$ 。在一些 branch normalization 中, 需要去掉某个 X^t 因子。由于在 R_N 中 $X^N = -1$, 有 $X^{-t} = -X^{N-t}$ 。因此实现中可能出现 $\text{MulPt}_{-X^{N-t}}$, 它表示乘以 X^{-t} 。例如若奇分支以 $XB(X^2)$ 的形式出现, 则可用 $\text{MulPt}_{-X^{N-1}}$ 将其归一化为 $B(X^2)$ 的语义。

Homomorphic linear transform. C2S、S2C、slot-preserving split 中的 $A \pm hB$ 、以及一般 slot selection $q = F_n^{-1}TF_Np$, 都可以看作 homomorphic linear transforms。实现中它们通常由 rotations、plaintext diagonal multiplications、rescale 和 key switching 组成。因此, 即使某个变换在明文线性代数中很自然, 在密文层面也可能代价昂贵。

Key switching. Key switching 将一个 secret 下的 ciphertext 转换到另一个 secret 下, 同时保持消息语义不变并加入小的 switching error。普通 rotation、relinearization、bootstrapping key、以及本文的 ring switching 都依赖 key switching。本文后文将把这些误差统一纳入 correctness 和 parameter analysis, 而在主理论等式中暂时忽略。

例如 1 (用 Aut_{1+2n} 提取偶部). 设 $F(X) = A(X^2) + XB(X^2) \in R_{2n}$ 。因为 Aut_{1+2n} 实现 $X \mapsto -X$, 有 $\text{Aut}_{1+2n}(F) = F(-X) = A(X^2) - XB(X^2)$ 。因此

$$\frac{F(X) + F(-X)}{2} = A(X^2), \quad \frac{F(X) - F(-X)}{2} = XB(X^2).$$

如果希望得到 $B(X^2)$, 还需要乘以 $X^{-1} = -X^{2n-1}$ 。密文层面, 这对应 *automorphism*、加法和一次 *plaintext multiplication*。这个例子说明: 即使 *even/odd split* 的明文公式很简单, 同态实现仍然包含 *rotations/key switching* 和明文乘法。

2.4 Ring switching 与 coefficient/module decomposition

令 $N = kn$, 并将 $R_{q,n}$ 视为由 $Y = X^k$ 诱导的子环。明文层面上, 任意 $P(X) \in R_{q,N}$ 都有唯一 decomposition:

$$P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k), \quad P_r(Y) \in R_{q,n}.$$

这给出了 coefficient/module split: 大环作为小环上的 rank- k module, 可以按 $1, X, \dots, X^{k-1}$ 分解。系数层面, 它就是按 index modulo k 的 residue classes 分组。

但是在 ciphertext level, 不能简单地重排 c_0, c_1 的系数后就得到小环 ciphertext。原因是 CKKS ciphertext 的消息通过 $c_0 + c_1 s$ 隐含在 secret-dependent 解密关系中。若要得到真正的小环 ciphertext, 必须把 secret 语义也切换到小环。本文把这种 lower-degree conversion 抽象写作

$$\text{RSDown}_{N \rightarrow n}(\mathbf{ct}_N) = (\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1}),$$

其理想消息语义为: 若 $\mathbf{ct}_N \models_{\Delta, N} P(X)$, 且 $\text{Split}_k(P) = (P_0, \dots, P_{k-1})$, 则 $\mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y)$ 。

反向地, upper-degree conversion 将 k 个小环 ciphertexts 合并回大环 ciphertext。若 $\mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y)$, 则理想的 $\text{RSUp}_{n \rightarrow N}$ 输出满足

$$\text{RSUp}_{n \rightarrow N}(\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1}) \models_{\Delta, N} \sum_{r=0}^{k-1} X^r P_r(X^k).$$

概念上, 可以把 RSDown 看成“key switching + split”, 把 RSUp 看成“merge + key switching”。具体顺序依赖实现。例如某些实现会先构造 module ciphertext, 再进行 switching; 也有实现把 decomposition 和 switching 组织成一个整体 evaluator。本文理论上只需要它们满足上述消息语义。

注 3 (ring switching 与 Section 3 的关系). Section 3 将详细解释 $\text{Split}_k(P)$ 在 coefficient view 和 slot view 下分别意味着什么。本小节只强调 ciphertext-level 事实: RSDown 和 RSUp 是带 key-switching noise 的密文转换, 不是免费明文重排。

2.5 Coefficient split 与 slot-preserving split 的最小区别

为了避免与 Section 3 重复，这里只给出最小区分。对于 $k = 2$ ，coefficient split 写作

$$P(X) = A(X^2) + XB(X^2).$$

若 $\theta = \alpha^2$ ，则一对 big-ring slots 满足

$$P(\alpha) = A(\theta) + \alpha B(\theta), \quad P(-\alpha) = A(\theta) - \alpha B(\theta).$$

因此 $A(\theta)$ 和 $B(\theta)$ 不是原始 slots $P(\alpha), P(-\alpha)$ 的直接选择，而是它们的局部线性组合。

如果目标是 slot-preserving split，即构造小环消息 $Q^+(Y), Q^-(Y)$ ，使得 $Q^+(\theta) = P(\alpha)$ 且 $Q^-(\theta) = P(-\alpha)$ ，则需要额外的 selector $h(Y)$ ，满足 $h(\theta) = \alpha$ 。于是

$$Q^+(Y) = A(Y) + h(Y)B(Y), \quad Q^-(Y) = A(Y) - h(Y)B(Y).$$

在密文层面，这意味着 coefficient split 之后还要执行额外的 homomorphic linear transform。本文的 bootstrapping 分解不采用该 slot-preserving 路线；我们使用 coefficient/module split，并证明它与 C2S-EvalMod-S2C core 兼容。

2.6 RLWE 安全性与参数选择

CKKS 的底层安全性来自 RLWE。粗略地，攻击难度主要受 ring degree N 、最大公开模数 $q_{\max}$ 、secret distribution、error distribution 和公开样本数量影响。在工程参数选择中，最重要的直觉是：

$$N \uparrow \Rightarrow \text{安全预算增加}, \quad q_{\max} \uparrow \Rightarrow \text{安全性下降}.$$

这里的 $q_{\max}$ 不应只理解为当前 ciphertext 剩余的 modulus。对于带 evaluation keys 的 CKKS bootstrapping 系统，公开材料可能包含 rotation keys、relinearization keys、bootstrapping keys、ring switching keys 等，它们常常在 QP 上生成。因此保守估计时应取

$$q_{\max} = \max\{Q_{\text{ct}}, Q_{\text{eval}}, QP_{\text{switch}}, QP_{\text{bts}}, QP_{\text{rs}}\}.$$

在初步工程分析中，可以先用 $q_{\max} \approx QP$ 估计。

本文的子环分解还带来一个额外安全点：如果 leaf ciphertexts 或 leaf evaluation keys 真正在 $R_{q,n}$ 上存在，那么它们的安全性必须按 leaf ring degree n 来估计，不能直接继承 top ring degree N 。因此，整体方案安全性应写为各组件安全性的最小值：

$$\lambda_{\text{scheme}} = \min\{\lambda_{\text{top}}, \lambda_{\text{leaf}}, \lambda_{\text{RSDown}}, \lambda_{\text{RSUp}}, \lambda_{\text{BTSkeys}}\}.$$

如果某个 leaf key 或 switching key 的 RLWE 参数只有较低安全级别，则整个方案也只能声称该较低级别。

例如 2 (为什么 leaf 安全要单独看). 假设 top ring 取 $\log N = 17$ ，并使用 $k = 2$ ，则 leaf ring 为 $\log n = 16$ 。如果 bootstrapping 和 switching keys 的最大公开模数约为 $\log_2(QP) \approx 1520$ ，那么需要检查的是 $n = 2^{16}$ 是否能承受这一 $q_{\max}$ ，而不是只检查 top $N = 2^{17}$ 。如果改用 $k = 4$ 且 top 仍为 $\log N = 16$ ，则 leaf 只有 $\log n = 14$ ，同样的 QP 会远超常见 128-bit 参考预算；这种设置更适合定位为 correctness prototype，而不是安全实例。

[待完成：最终论文需要给出 lattice estimator 结果，并分别列出 top ciphertext/key、leaf bootstrapping keys、RSDown/RSUp keys 的 N 、 $q_{\max}$ 、secret distribution 和 estimated security bits。]

3 子环分解的 Coefficient/Slot 语义

本节解释本文所用子环分解的语义。需要首先强调的是：在同态算法中，我们实际持有和操作的对象始终是密文 $\mathbf{ct}$ ，而不是明文多项式 $P(X)$ 、系数向量 p ，也不是 slot 向量。明文多项式、系数向量和 slot 向量只用于描述密文解密后的消息语义。因此，本节关于 coefficient split、slot split 和 slot-preserving split 的讨论，都应理解为密文所表示消息的语义，而不是明文可见操作。

本节采用 full complex canonical embedding 来说明主要关系。实际 CKKS 实现通常只使用半槽，并包含 conjugation、bit-reversal、 5^i root ordering 等 layout convention；这些差异在本文理论层面都可以吸收到固定的 layout permutation 中。实现层面的 layout 和 normalization 将在实验章节单独讨论。

3.1 密文表示语义：我们实际操作的是 $\mathbf{ct}$

设当前 CKKS scale 为 Δ 。若一个大环密文 $\mathbf{ct}_N = (c_0, c_1) \in \mathcal{R}_{N,q}^2$ 在私钥 s_N 下满足

$$c_0 + c_1 s_N = \text{Round}(\Delta P(X)) + e \pmod{q},$$

其中 $P(X) \in \mathcal{R}_N$ 是被表示的消息多项式， e 是 CKKS 噪声项，则记作

$$\mathbf{ct}_N \models_{\Delta, N} P(X).$$

这里 $\models_{\Delta, N}$ 表示“在大环 $\mathcal{R}_N$ 和 scale Δ 下表示”。类似地，若小环密文 $\mathbf{ct}_n \in \mathcal{R}_{n,q}^2$ 在小环私钥 s_n 下表示 $Q(Y) \in \mathcal{R}_n$ ，则记作 $\mathbf{ct}_n \models_{\Delta, n} Q(Y)$ 。

如果从 CKKS 编码视角出发，一个 slot 向量 $\mathbf{z}$ 会先通过编码映射变成一个消息多项式。例如在抽象记号下可写为 $\text{Ecd}(\mathbf{z}) = \text{Round}(\Delta \cdot \tau^{-1}(\mathbf{z}))$ 。所以当我们写 $\mathbf{ct} \models_{\Delta, N} P(X)$ 时，也可以理解为该密文加密了某个 slot 向量经过 CKKS 编码后得到的多项式消息。本文后续使用 $P(X)$ 、 $Q(Y)$ 来描述消息语义，是为了避免在每个公式中都展开编码和取整。

注 4 (消息语义与密文操作). 当我们写 $\text{Split}_k(P) = (P_0, \dots, P_{k-1})$ 时，这只是明文语义层面的分解。真正的密文级操作应当是：给定 $\mathbf{ct}_N \models_{\Delta, N} P(X)$ ，通过 ring switching 或 key switching 产生 leaf ciphertexts $\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1}$ ，并使得 $\mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y)$ 。这个转换不是免费重排，而是会引入 switching noise 和相应的 switching keys。

3.2 Coefficient/module split：密文下的目标语义

令 $N = kn$ ，并令 $\mathcal{R}_N = \mathbb{C}[X]/(X^N + 1)$ 、 $\mathcal{R}_n = \mathbb{C}[Y]/(Y^n + 1)$ 。通过代换 $Y = X^k$ ，任意 $P(X) \in \mathcal{R}_N$ 都可以唯一写成

$$P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k), \quad P_r(Y) \in \mathcal{R}_n.$$

如果 $P(X) = \sum_{i=0}^{N-1} a_i X^i$, 则 $P_r(Y) = \sum_{j=0}^{n-1} a_{r+jk} Y^j$ 。也就是说, 第 r 个小环多项式 P_r 收集的是大环消息多项式中所有下标同余于 $r \pmod k$ 的系数。

定义 1 (Coefficient/module split). 明文层面的 coefficient/module split 定义为

$$\text{Split}_k(P) = (P_0, \dots, P_{k-1}), \quad P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k).$$

对应的 merge map 定义为

$$\text{Merge}_k(P_0, \dots, P_{k-1}) = \sum_{r=0}^{k-1} X^r P_r(X^k).$$

把这个定义放回密文语义中, coefficient split 的目标是: 给定 $\mathbf{ct}_N \models_{\Delta, N} P(X)$, 构造 k 个小环密文 $\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1}$, 使得

$$\mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y), \quad r = 0, \dots, k-1.$$

这才是本文所谓“把大环密文拆到小环”的含义。输入密文仍然只是一对大环多项式 (c_0, c_1) , 输出应当是 k 个新的小环密文, 并且每个输出密文在小环私钥语义下解密为对应的 $P_r(Y)$ 。

这里不能把 coefficient split 误解为“对密文分量 c_0, c_1 直接取 residue class”。原因是 c_0, c_1 本身并不等于消息 P 的系数; 消息只通过解密方程 $c_0 + c_1 s_N \approx \Delta P$ 隐含在密文中。如果只对 c_0, c_1 做公开系数抽取, 通常得到的是绑定到大环 secret 的 module relation, 而不是普通小环私钥下的 CKKS ciphertext。因此, 真正的 ciphertext-level split 必须包含 ring switching 或 key switching。本文后文用 $\text{RSDown}_{N \rightarrow n}$ 表示该密文级操作, 其理想语义是

$$\text{RSDown}_{N \rightarrow n}(\mathbf{ct}_N) = (\mathbf{ct}_{n,0}, \dots, \mathbf{ct}_{n,k-1}), \quad \mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y).$$

为了描述 P_r 在系数层面的结构, 令 $p = (a_0, \dots, a_{N-1})^T$ 为 P 的系数向量。定义 residue-class 选择矩阵 $S_r : \mathbb{C}^N \rightarrow \mathbb{C}^n$ 为

$$S_r p = (a_r, a_{r+k}, a_{r+2k}, \dots, a_{r+(n-1)k})^T.$$

于是 P_r 的系数向量就是 $S_r p$ 。把所有 S_r 合在一起, 记 $S_k p = (S_0 p, \dots, S_{k-1} p)$ 。在消息系数语义下, Split_k 对应 $p \mapsto S_k p$; 但在密文层面, 要实现这一语义仍然需要 $\text{RSDown}_{N \rightarrow n}$ 。

例如 3 ($k=2$ 时的二维 coefficient split). 考虑 $N=2n$ 。任意 $P(X) \in \mathcal{R}_N$ 都可以唯一写成

$$P(X) = A(X^2) + XB(X^2),$$

其中 $A(Y), B(Y) \in \mathcal{R}_n$ 。若 $P(X) = \sum_{i=0}^{2n-1} a_i X^i$, 则

$$A(Y) = \sum_{j=0}^{n-1} a_{2j} Y^j, \quad B(Y) = \sum_{j=0}^{n-1} a_{2j+1} Y^j.$$

因此, 二维 coefficient split 在消息语义上就是 $P \mapsto (A, B)$ 。

在密文语义下，这句话应理解为：若输入大环密文满足 $\mathbf{ct}_N \models_{\Delta, N} P(X)$ ，那么理想的 *ring-switch-down* 输出应是两个小环密文 $\mathbf{ct}_A, \mathbf{ct}_B$ ，使得

$$\mathbf{ct}_A \models_{\Delta, n} A(Y), \quad \mathbf{ct}_B \models_{\Delta, n} B(Y).$$

也就是说，*split* 后我们希望得到的不是两个可见明文 A, B ，而是两个分别表示 A, B 的 *leaf ciphertexts*。

该二维分解对加法是逐分量的。若 $P(X) = A(X^2) + XB(X^2)$ 且 $Q(X) = C(X^2) + XD(X^2)$ ，则

$$P(X) + Q(X) = (A + C)(X^2) + X(B + D)(X^2).$$

但是该分解对乘法不是逐分量的。直接展开可得

$$\begin{aligned} P(X)Q(X) &= (A(X^2) + XB(X^2))(C(X^2) + XD(X^2)) \\ &= (AC)(X^2) + X(AD + BC)(X^2) + X^2(BD)(X^2). \end{aligned}$$

令 $Y = X^2$ ，上式可以重新写成

$$P(X)Q(X) = (AC + YBD)(X^2) + X(AD + BC)(X^2).$$

所以乘法后的偶分支不是 AC ，而是 $AC + YBD$ ；奇分支也不是 BD ，而是 $AD + BC$ 。这说明 *coefficient split* 给出的是一个二维 *module decomposition*，而不是把大环乘法变成 *leaf-wise multiplication* 的 *direct-product decomposition*。

一般 k 的情况与上述二维例子相同： $\mathbf{Split}_k$ 是一个线性 *module decomposition*，与加法兼容，但乘法会在不同 *residue components* 之间产生交叉项。因此，本文使用 *coefficient split* 的原因不是为了把所有同态运算都变成 *leaf-wise independent computation*，而是因为 CKKS bootstrapping core 具有特殊结构。更具体地说，bootstrapping core 可以写成“线性变换 C2S、逐坐标非线性 EvalMod、反向线性变换 S2C”的形式；*coefficient split* 与前后两个线性变换兼容，中间的 EvalMod 又是逐坐标作用。这才是后文能够证明 *leaf-wise bootstrapping factorization* 的原因。

3.3 子环嵌入与压缩的 slot 几何

在讨论一般 *coefficient split* 的 slot 语义之前，先看一个更简单的特殊情况：消息本来就在子环嵌入像中。设 $N = 2n$ ，小环消息为 $f(Y) \in \mathcal{R}_n$ ，其嵌入到大环后的消息为 $f(X^2) \in \mathcal{R}_{2n}$ 。若小环 slot vector 为 $z = (f(\theta_0), \dots, f(\theta_{n-1}))$ ，那么大环中 $f(X^2)$ 的 slot values 是 $f(\alpha_i^2)$ 。

在成对排列下， $\alpha_{j+n} = -\alpha_j$ ，且 $\alpha_j^2 = \alpha_{j+n}^2 = \theta_j$ 。因此

$$f(\alpha_j^2) = f(\theta_j), \quad f(\alpha_{j+n}^2) = f(\theta_j).$$

所以大环 slot vector 具有重复结构：

$$\text{Slots}_{2n}(f(X^2)) = (z, z),$$

这里等式忽略了实现中的固定 *layout permutation*。相反，从 $f(X^2)$ 压缩回 $f(Y)$ 在 slot 层面就是

$$(z, z) \longmapsto z.$$

例如 4 (子环嵌入导致重复 slots). 取 $n = 2, N = 4$, 令 $f(Y) = b_0 + b_1 Y$. 设小环 roots 为 θ_0, θ_1 , 大环 roots 成对为 $\alpha_0, -\alpha_0$ 和 $\alpha_1, -\alpha_1$, 其中 $\alpha_j^2 = \theta_j$. 则

$$f(X^2)(\alpha_j) = f(\theta_j), \quad f(X^2)(-\alpha_j) = f(\theta_j).$$

因此大环中每一对 *paired slots* 完全相同。这说明 $f(Y) \mapsto f(X^2)$ 不是把信息变复杂, 而是在大环中重复存放同一份小环 slot 信息。

这个特殊情况对理解 ring compression 很有帮助。如果输入密文满足 $\mathbf{ct}_{2n} \models_{\Delta, 2n} f(X^2)$, 那么理想的压缩输出应满足 $\mathbf{ct}_n \models_{\Delta, n} f(Y)$. 在 slot 语义上, 它把重复结构 (z, z) 压回一份 z . 如果输入噪声写成 $e(X) = e_0(X^2) + X e_1(X^2)$, 那么压缩后保留的主要是偶部噪声 e_0 , 并额外加入 key-switching noise. 这可以直观理解为: 压缩保留 *paired slots* 的对称部分, 丢弃反对称部分。

3.4 Coefficient split 在 slot 视角下发生什么

现在回到一般消息 $F(X) \in \mathcal{R}_{2n}$. 写成

$$F(X) = A(X^2) + X B(X^2).$$

设 $\theta_j = \alpha_j^2$, 并记 $a_j = A(\theta_j)$ 、 $b_j = B(\theta_j)$. 则成对的大环 slots 满足

$$F(\alpha_j) = a_j + \alpha_j b_j, \quad F(-\alpha_j) = a_j - \alpha_j b_j.$$

因此, coefficient split 输出的 $A(Y)$ 和 $B(Y)$ 在小环 slots 中对应

$$a_j = \frac{F(\alpha_j) + F(-\alpha_j)}{2}, \quad b_j = \frac{F(\alpha_j) - F(-\alpha_j)}{2\alpha_j}.$$

这说明 coefficient split 在 slot 视角下不是选择原始大环 slots, 而是在每一对 *paired slots* 上做局部 butterfly: 它提取对称部分 a_j , 以及归一化后的反对称部分 b_j .

命题 1 (coefficient split 不是 slot selection). 对于 $N = 2n$, coefficient split $F(X) = A(X^2) + X B(X^2)$ 在 slot 语义下将一对 *big-ring slots* $(F(\alpha_j), F(-\alpha_j))$ 变为 $(A(\theta_j), B(\theta_j))$. 该变换为局部线性变换, 而不是坐标选择。

证明. 由 $F(\alpha_j) = A(\theta_j) + \alpha_j B(\theta_j)$ 与 $F(-\alpha_j) = A(\theta_j) - \alpha_j B(\theta_j)$, 相加得到 $A(\theta_j) = (F(\alpha_j) + F(-\alpha_j))/2$, 相减得到 $B(\theta_j) = (F(\alpha_j) - F(-\alpha_j))/(2\alpha_j)$. $\square$

一般 k 的情况可以按 fiber 理解。固定一个小环 root θ , 取其上方的 k 个大环 roots $\alpha_{\theta,0}, \dots, \alpha_{\theta,k-1}$, 满足 $\alpha_{\theta,\ell}^k = \theta$. 由 $P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k)$, 对每个 ℓ 有

$$P(\alpha_{\theta,\ell}) = \sum_{r=0}^{k-1} \alpha_{\theta,\ell}^r P_r(\theta).$$

记 $z_\theta = (P(\alpha_{\theta,0}), \dots, P(\alpha_{\theta,k-1}))^T$, $u_\theta = (P_0(\theta), \dots, P_{k-1}(\theta))^T$. 则

$$z_\theta = V_\theta u_\theta, \quad (V_\theta)_{\ell,r} = \alpha_{\theta,\ell}^r.$$

因此 $u_\theta = V_\theta^{-1} z_\theta$. 这说明一般 k 下, coefficient split 在 slot 视角中对应每个 fiber 上的局部 Vandermonde 逆变换。

例如 5 (从 paired slots 到偶/奇分支). 仍取 $N = 2n$. 如果某一对大环 slots 为 $(u + \alpha v, u - \alpha v)$, 那么 *coefficient split* 后的两个小环分支在对应小环 slot 上就是 u 和 v . 如果输入本来是子环嵌入 $f(X^2)$, 则 $v = 0$, *paired slots* 退化为 (u, u) . 这正是上一小节中重复结构 (z, z) 的特殊情形。

3.5 Trace-like projection 的补充视角

上面的 even/odd decomposition 也可以用 trace-like projection 来理解。对于 $N = 2n$, 固定小环 $\mathbb{C}(X^2)$ 的非平凡自同构可以理解为 $X \mapsto -X$. 因此

$$\frac{F(X) + F(-X)}{2} = A(X^2), \quad \frac{F(X) - F(-X)}{2X} = B(X^2).$$

第一式提取偶部, 第二式提取归一化后的奇部。

例如 6 (trace 提取偶部). 设 $F(X) = a_0 + a_1X + a_2X^2 + a_3X^3$. 则

$$\frac{F(X) + F(-X)}{2} = a_0 + a_2X^2 = A(X^2),$$

而

$$\frac{F(X) - F(-X)}{2X} = a_1 + a_3X^2 = B(X^2).$$

这说明 *coefficient split* 的 A, B 可以看成由对 $X \mapsto -X$ 的对称/反对称投影得到。

该视角有助于理解“为什么偶部落在小环里”。不过本文的主算法并不是直接把 trace 作为密文操作, 而是通过 HERMES-style ring switching 获得 leaf ciphertexts. 因此, 本小节只作为语义解释。

3.6 Slot split 与 slot-preserving split

与 coefficient split 相对, slot split 的目标是直接控制 leaf ciphertexts 的 slot 语义。给定 $\mathbf{ct}_N \models_{\Delta, N} P(X)$, 其 big-ring slot vector 可写为 $z = F_N p$, 其中 p 是 P 的系数向量, F_N 是大环 evaluation matrix. 如果我们希望某个小环密文 $\mathbf{ct}_q$ 表示 $Q(Y)$, 并且 Q 的小环 slot vector 是指定的 $y = Tz$, 那么 Q 的系数向量 q 必须满足 $F_n q = y$. 因此

$$q = F_n^{-1} T F_N p.$$

这就是一般 slot split 的系数形式。这里 T 可以是 slot selection, 也可以是任意 slot-level linear transform.

这个公式说明: slot split 不是简单的 coefficient residue-class selection. 除非 T 与 F_N, F_n 有特殊结构, 否则 $F_n^{-1} T F_N$ 通常是稠密线性变换。在密文层面, 它意味着需要执行额外的 homomorphic linear transform. 因此, slot split 更通用, 但通常更贵。

Slot-preserving split 是 slot split 的一个重要特例。仍以 $N = 2n$ 为例。我们希望构造两个小环多项式 $Q^+(Y)$ 和 $Q^-(Y)$, 使得

$$Q^+(\theta_j) = F(\alpha_j), \quad Q^-(\theta_j) = F(-\alpha_j).$$

而 coefficient split 只给出 $A(Y)$ 和 $B(Y)$ 。由 $F(\alpha_j) = A(\theta_j) + \alpha_j B(\theta_j)$ 可知, 如果存在小环多项式 $h(Y)$ 满足 $h(\theta_j) = \alpha_j$, 则可定义

$$Q^+(Y) = A(Y) + h(Y)B(Y), \quad Q^-(Y) = A(Y) - h(Y)B(Y).$$

于是 $Q^+(\theta_j) = F(\alpha_j)$, 且 $Q^-(\theta_j) = F(-\alpha_j)$ 。

在密文层面, 如果 $\mathbf{ct}_A \models_{\Delta, n} A(Y)$ 和 $\mathbf{ct}_B \models_{\Delta, n} B(Y)$, 则 slot-preserving split 的第二步需要计算

$$\mathbf{ct}^+ = \mathbf{ct}_A + \text{MulPt}_{h(Y)}(\mathbf{ct}_B), \quad \mathbf{ct}^- = \mathbf{ct}_A - \text{MulPt}_{h(Y)}(\mathbf{ct}_B).$$

这一步不是 coefficient split 本身的一部分, 而是额外的 leaf-side homomorphic linear transform。

用矩阵形式表示, 令 E 与 O 分别为偶、奇系数选择矩阵, 令 $D = \text{diag}(\alpha_0, \dots, \alpha_{n-1})$, 并令 $H = F_n^{-1} D F_n$ 。则 coefficient split 是 $p \mapsto (Ep, Op)$, 而 pairwise slot-preserving split 是

$$p \mapsto (Ep + HOp, Ep - HOp).$$

因此, slot-preserving split 比 coefficient split 多了一个 H , 也就是小环上乘以平方根选择多项式 $h(Y)$ 的线性变换。

例如 7 (slot-preserving split 的额外代价). 如果 $F(X) = A(X^2) + XB(X^2)$, coefficient split 只输出表示 A 和 B 的两个 leaf ciphertexts。若希望两个 leaf ciphertexts 的 slots 分别等于 $F(\alpha_j)$ 和 $F(-\alpha_j)$, 则还必须从 A, B 构造 $A + hB$ 与 $A - hB$ 。这一步在明文语义上很简单, 但在密文层面需要额外的明文乘法和加减法; 一般 slot selection $q = F_n^{-1} T F_N p$ 则可能对应更稠密的同态线性变换。

3.7 Slot-level split as representation switching

还有一种常见的 slot-level split 来自 CKKS 的表示切换。它不是按 $Y = X^k$ 做 ring switching, 而是在同一个 CKKS 编码语义下, 把一个 slot 向量中携带的系数信息同态地分成两个密文。

设

$$p(Y) = \sum_{i=0}^{2n-1} p_i Y^i, \quad \mathbf{p}_0 = (p_0, \dots, p_{n-1}), \quad \mathbf{p}_1 = (p_n, \dots, p_{2n-1}).$$

定义复向量 $\mathbf{w} = \mathbf{p}_0 + \mathbf{i} \mathbf{p}_1$, 并令 $\mathbf{z} = U_n \mathbf{w}$ 。在 CKKS 编码语义下, 一个密文可以表示 $\text{Ecd}(\mathbf{z})$, 也就是表示由 $p(Y)$ 编码得到的消息。若对该密文同态乘以 U_n^{-1} , 则 slot 语义变为 $\mathbf{p}_0 + \mathbf{i} \mathbf{p}_1$ 。随后通过共轭可分离实部和虚部:

$$\frac{1}{2}(\mathbf{w} + \bar{\mathbf{w}}) = \mathbf{p}_0, \quad \frac{1}{2\mathbf{i}}(\mathbf{w} - \bar{\mathbf{w}}) = \mathbf{p}_1.$$

密文层面相应地得到两个密文, 分别表示 $\text{Ecd}(\mathbf{p}_0)$ 和 $\text{Ecd}(\mathbf{p}_1)$ 。

反向的 SlotToCoeff 则从两个密文开始: 一个表示 $\text{Ecd}(\mathbf{p}_0)$, 另一个表示 $\text{Ecd}(\mathbf{p}_1)$ 。先同态构造 $\mathbf{p}_0 + \mathbf{i} \mathbf{p}_1$, 再乘以 U_n , 得到表示 $\text{Ecd}(\mathbf{z})$ 的密文。这个过程可以理解为把“slot 中装着系数信息”的表示切换回“一个多项式被编码”的表示。

例如 8 (一个密文拆成两个 slot 密文). 假设输入密文表示 $\text{Ecd}(\mathbf{z})$, 其中 $\mathbf{z} = U_n(\mathbf{p}_0 + \mathbf{i}\mathbf{p}_1)$. 同态乘 U_n^{-1} 后, 密文表示 $\text{Ecd}(\mathbf{p}_0 + \mathbf{i}\mathbf{p}_1)$. 再做

$$\mathbf{ct}_0 = \frac{1}{2}(\mathbf{ct}' + \text{Conj}(\mathbf{ct}')), \quad \mathbf{ct}_1 = \frac{1}{2\mathbf{i}}(\mathbf{ct}' - \text{Conj}(\mathbf{ct}')),$$

即可分别得到表示 $\mathbf{p}_0$ 和 $\mathbf{p}_1$ 的两个密文。这个过程是 *slot-level representation switching*, 而不是本文主算法中的 *HERMES-style ring switching*。

这类 slot-level split 对解释 CKKS 的 CoeffToSlot/SlotToCoeff 很有用, 但它与本文的 coefficient/module split 不同。本文的目标是把一个大环 ciphertext 转换为多个小环 ciphertexts, 使 leaf ciphertexts 分别表示 $P_r(Y)$ 。而 slot-level split 通常仍是在编码语义中做同态线性变换, 并不自动改变 ciphertext 所在的环维度。

3.8 Two-step splitting interpretation

上述几种 split 可以统一为一个 two-step view。

第一步是 ciphertext-level coefficient/module decomposition。输入是大环密文 $\mathbf{ct}_N \models_{\Delta, N} P(X)$ 。通过 ring switching, 输出 k 个 leaf ciphertexts $\mathbf{ct}_{n,r}$, 并使得 $\mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y)$ 。这一步改变了密文所在的环和密钥语义, 是本文加速的基础。

第二步是可选的 slot-coordinate transformation。如果下游任务要求 leaf ciphertexts 的 slots 具有特定的 big-ring slot 语义, 则需要在第一步之后继续执行额外同态线性变换。对于 $k = 2$, 从 coefficient split 到 pairwise slot-preserving split 的第二步是 $A, B \mapsto A \pm hB$ 。对于一般 slot selection, 它由 $q = F_n^{-1}TF_Np$ 描述。对于 CKKS 表示切换场景, 它也可能表现为 U_n^{-1} 、conjugation、 U_n 这一类 slot-level linear transforms。

本文的 bootstrapping 分解只使用第一步, 不执行第二步。原因是 CKKS bootstrapping core 并不要求 split 后立即恢复原始 big-ring slots。它需要的是: 经过 C2S 后, slots 中装入 coefficient-like values, 然后 EvalMod 对这些 values 逐坐标作用。Coefficient split 后直接接 leaf C2S_n, 恰好得到大环 C2S_N 输出的 coefficient residue-class 子序列。因此, slot-preserving transformation 不是本文主算法的一部分。

3.9 为什么 bootstrapping 分解选择 coefficient split

现在可以解释本文为什么选择 coefficient split, 而不是 slot-preserving split。

CKKS bootstrapping core 的逻辑不是“在 split 后保留原始 big-ring slots 并继续计算”。传统 core 的结构是先通过 C2S 把待模约减的 coefficient-like values 放入 slots, 然后用 EvalMod 逐坐标去掉 q_0I_i , 最后通过 S2C 回到系数表示。因此, 分解后真正需要对齐的是 C2S 之后的 coefficient-packed coordinates, 而不是 split 之后的 canonical slots。

Coefficient split 虽然在 canonical slot 视角下会混合 big-ring slots, 但它有一个关键优点: 若先对密文执行 ring-switch-down 得到 $\mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y)$, 再对每个 leaf 执行 C2S_n, 则 leaf 的 C2S_n 输出正是 P_r 的系数, 也就是大环 P 的 residue-class 系数子序列。换句话说, $\text{Split}_k + \bigoplus_r \text{C2S}_n$ 在语义上等价于先做 top C2S_N, 再把输出按 coefficient residue classes 分组。

这解释了为什么本文不需要先做 slot-preserving split。Slot-preserving split 试图恢复 $P(\alpha)$ 、 $P(-\alpha)$ 这类 big-ring slot values; 但 bootstrapping 的 EvalMod 阶段需要处理的是 C2S 后的

coefficient-packed values。额外恢复 big-ring slots 不仅不是必要步骤，还会引入额外线性变换成本。

3.10 本节小结

本节的结论可以概括为四点。第一，本文讨论的 split 是密文语义下的 split：给定 $\mathbf{ct}_N \models_{\Delta, N} P(X)$ ，目标是通过 ring switching 得到 $\mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y)$ ，而不是直接读取 P 或其系数。第二，coefficient/module split 在系数层面是简单的 residue-class decomposition，但在 slot 视角下不是 slot selection；对于 $k = 2$ ，它提取 paired slots 的平均部分和加权差值。第三，如果希望拆分后保留原始 big-ring slot 语义，就需要 slot split 或 slot-preserving split，这通常需要额外同态线性变换，例如 $A \pm hB$ 或更一般的 $F_n^{-1}TF_N$ 。第四，本文的 bootstrapping factorization 选择 coefficient split，是因为 Split_k 后接 leaf C2S_n 正好获得 top C2S_N 输出的 coefficient residue-class 子序列。下一节将把这一点形式化为 Split-C2S compatibility，并进一步证明 EvalMod 与 S2C + Merge 的兼容性。

3.11 本节小结

本节的结论可以总结如下。

第一，本文讨论的 split 是密文语义下的 split。给定 $\mathbf{ct}_N \models_{\Delta, N} P(X)$ ，目标是通过 ring switching 得到 $\mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y)$ ，而不是直接读取 P 或其系数。

第二，coefficient/module split 在系数层面是简单的 residue-class decomposition： $p \mapsto (S_0p, \dots, S_{k-1}p)$ 。但在 slot 视角下，它不是 slot selection；对于 $k = 2$ ，它把 $(P(\alpha), P(-\alpha))$ 变成 $(A(\theta), B(\theta))$ ，即平均和加权差值；对于一般 k ，它对应每个 fiber 上的局部 Vandermonde 逆变换。

第三，如果希望拆分后保留原始 big-ring slot 语义，就需要 slot split 或 slot-preserving split。这通常需要额外的同态线性变换，例如 $A \pm hB$ ，或更一般的 $F_n^{-1}TF_N$ 。

第四，本文的 bootstrapping factorization 选择 coefficient split，是因为 Split_k 后接 leaf C2S_n 正好获得 top C2S_N 输出的 coefficient residue-class 子序列。下一节将把这一点形式化为 Split-C2S compatibility，并进一步证明 EvalMod 与 S2C + Merge 的兼容性。

4 主要理论结果

上一节说明了本文使用的 split 是密文语义下的 coefficient/module split：若输入密文满足 $\mathbf{ct}_N \models_{\Delta, N} P(X)$ ，则理想的 ring-switch-down 输出应满足 $\mathbf{ct}_{n,r} \models_{\Delta, n} P_r(Y)$ ，其中 $P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k)$ 。本节在消息语义和 normalized logical layout 下证明本文的核心代数结论：传统 CKKS bootstrapping core 可以沿该 coefficient split 分解为多个 leaf bootstrapping core。

为避免实现细节干扰，本节暂时忽略 Lattigo 中的 bit-reversal、blocked layout、real/imag branches、half-slot convention 和 scaling convention。所有这些实现层细节都可以视作固定 layout permutation 或 implementation scaling，并在实现章节单独讨论。本节只证明理想语义下的结构性等式。

4.1 Notations

固定 $N = kn$ 。令 Can_d 表示环维度为 d 时的 canonical-slot 坐标空间，令 Coeff_d 表示对应的 coefficient-packed 坐标空间。若 $P \in \mathcal{R}_d$ ，记 $\text{can}_d(P) \in \text{Can}_d$ 为 P 的 canonical-slot 向量，记 $\text{coeff}_d(P) \in \text{Coeff}_d$ 为 P 的系数向量。

本节把 $\text{C2S}_d : \text{Can}_d \rightarrow \text{Coeff}_d$ 视为理想的 coefficient-to-slot 反向表示切换：它把 canonical-slot 坐标中的消息转为 EvalMod 所作用的 coefficient-packed 坐标。相应地， $\text{S2C}_d : \text{Coeff}_d \rightarrow \text{Can}_d$ 是反向线性变换。在 full canonical embedding 的 normalized model 下，可以把 C2S_d 理解为 evaluation matrix 的逆，把 S2C_d 理解为 evaluation matrix；但是本文的证明只使用二者的语义关系：

$$\text{C2S}_d(\text{can}_d(P)) = \text{coeff}_d(P), \quad \text{S2C}_d(\text{coeff}_d(P)) = \text{can}_d(P).$$

这里单独展示该式是因为它是后续所有交换图的语义基础。

对任意大环系数向量 $p = (a_0, \dots, a_{N-1})^T \in \text{Coeff}_N$ ，定义 residue-class 选择算子 $S_r : \text{Coeff}_N \rightarrow \text{Coeff}_n$ 为

$$S_r p = (a_r, a_{r+k}, a_{r+2k}, \dots, a_{r+(n-1)k})^T, \quad r = 0, \dots, k-1.$$

令 $S_k p = (S_0 p, \dots, S_{k-1} p)$ 。在 normalized layout 下， S_k 就是把长度为 N 的 coefficient-packed vector 按 residue class 分组为 k 个长度为 n 的 leaf vectors。

另一方面，上一节定义的明文 split 为

$$P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k), \quad \text{Split}_k(P) = (P_0, \dots, P_{k-1}).$$

在 canonical-slot 坐标中，我们也记 Split_k 为由该消息 split 诱导的语义 map：

$$\text{Split}_k(\text{can}_N(P)) = (\text{can}_n(P_0), \dots, \text{can}_n(P_{k-1})).$$

类似地， Merge_k 在 canonical-slot 语义下表示先将 leaf 消息 $P_0, \dots, P_{k-1}$ 组装成 $\sum_r X^r P_r(X^k)$ ，再取大环 canonical-slot 表示。

最后，用 $\Pi_k : \text{Coeff}_N \rightarrow \bigoplus_{r=0}^{k-1} \text{Coeff}_n$ 表示 top coefficient-packed vector 到 leaf coefficient-packed vectors 的 layout map。在 normalized logical layout 下， $\Pi_k = S_k$ 。在具体实现中， Π_k 还可能包含 bit-reversal、block ordering 或 real/imag branch ordering；这些都不影响本节的代数结论。

例如 9 (符号在 $k = 2$ 时的含义)。若 $N = 2n$ ，任意 $P(X)$ 可写成 $P(X) = A(X^2) + XB(X^2)$ 。若 $p = (a_0, a_1, a_2, a_3, \dots)^T$ ，则 $S_0 p = (a_0, a_2, a_4, \dots)^T$ 是 $A(Y)$ 的系数向量， $S_1 p = (a_1, a_3, a_5, \dots)^T$ 是 $B(Y)$ 的系数向量。因此 $\Pi_2 p = (S_0 p, S_1 p)$ 就是把 top coefficient-packed vector 分成偶系数分支和奇系数分支。

4.2 引理与定理陈述

本节的核心结论由三个兼容性引理组成。它们分别对应 CKKS bootstrapping core 的三段：C2S、EvalMod、S2C。直观上，第一条引理说明 split 后做 leaf C2S_n 等价于先做 top C2S_N 再分组；第二条引理说明逐坐标的 EvalMod 可以穿过这种分组；第三条引理说明 leaf S2C_n 后再 merge 等价于 top S2C_N。

引理 1 (Split-C2S compatibility). 对于 $N = kn$, 在上述 *normalized layout convention* 下, $\left(\bigoplus_{r=0}^{k-1} \text{C2S}_n\right) \circ \text{Split}_k = \Pi_k \circ \text{C2S}_N$.

引理 2 (EvalMod compatibility). 若 $\text{EvalMod}_N = f^{\oplus N}$ 、 $\text{EvalMod}_n = f^{\oplus n}$, 且二者由同一个 *coordinate-wise scalar function* f 定义, 则 $\left(\bigoplus_{r=0}^{k-1} \text{EvalMod}_n\right) \circ \Pi_k = \Pi_k \circ \text{EvalMod}_N$.

引理 3 (S2C-Merge compatibility). 在相同 *layout convention* 下, $\text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{S2C}_n\right) \circ \Pi_k = \text{S2C}_N$.

定理 1 (CKKS bootstrapping core factorization). 在上述 *normalized layout convention* 下, *CKKS bootstrapping core* 满足

$$\text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N = \text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{S2C}_n \circ \text{EvalMod}_n \circ \text{C2S}_n\right) \circ \text{Split}_k. \quad (1)$$

式 (1) 的意义是: 理想模型中, 一个 top-level bootstrapping core 可以被替换为 split、 k 个 leaf core、再 merge 的组合。注意, 该等式不是说 split 后 leaf canonical slots 保留了原始 big-ring slots; 上一节已经说明 coefficient split 在 slot 视角下通常是局部 Vandermonde 型变换。这里真正起作用的是: split 后接 leaf C2S_n 会进入 coefficient-packed coordinates, 而这些 coordinates 正是 top C2S_N 输出的 residue-class 子序列。

4.3 Split-C2S compatibility 的证明

证明. 任取一个大环消息多项式 $P(X) = \sum_{i=0}^{N-1} a_i X^i$, 记其大环 canonical-slot 向量为 $z = \text{can}_N(P)$, 系数向量为 $p = \text{coeff}_N(P) = (a_0, \dots, a_{N-1})^T$ 。根据 C2S_N 的语义定义, 有 $\text{C2S}_N(z) = p$ 。

现在写出 P 的 coefficient/module split:

$$P(X) = \sum_{r=0}^{k-1} X^r P_r(X^k), \quad P_r(Y) = \sum_{j=0}^{n-1} a_{r+jk} Y^j.$$

因此, 第 r 个 leaf 消息 P_r 的系数向量正是 $S_r p$ 。

先看左侧路径。输入 $z = \text{can}_N(P)$ 经过 Split_k 后, 按照定义得到

$$\text{Split}_k(z) = (\text{can}_n(P_0), \dots, \text{can}_n(P_{k-1})).$$

再对每个 leaf 应用 C2S_n , 第 r 个分支变为

$$\text{C2S}_n(\text{can}_n(P_r)) = \text{coeff}_n(P_r) = S_r p.$$

因此左侧整体输出为

$$\begin{aligned} \left(\left(\bigoplus_{r=0}^{k-1} \text{C2S}_n\right) \circ \text{Split}_k\right)(z) &= (\text{C2S}_n(\text{can}_n(P_0)), \dots, \text{C2S}_n(\text{can}_n(P_{k-1}))) \\ &= (\text{coeff}_n(P_0), \dots, \text{coeff}_n(P_{k-1})) \\ &= (S_0 p, \dots, S_{k-1} p) \\ &= S_k p. \end{aligned}$$

再看右侧路径。输入 z 先经过 top-level C2S_N ，得到 p 。随后 Π_k 将 p 按 residue class 分组。在 normalized layout 下， $\Pi_k p = S_k p$ 。所以右侧输出为

$$\begin{aligned} (\Pi_k \circ \text{C2S}_N)(z) &= \Pi_k(\text{C2S}_N(z)) \\ &= \Pi_k p \\ &= S_k p. \end{aligned}$$

左右两条路径对任意 P 的 canonical-slot 输入 z 都得到同一个 grouped coefficient vector $S_k p$ 。因此

$$\left(\bigoplus_{r=0}^{k-1} \text{C2S}_n \right) \circ \text{Split}_k = \Pi_k \circ \text{C2S}_N.$$

□

例如 10 ($k = 2$ 时的 Split-C2S compatibility). 令 $P(X) = A(X^2) + XB(X^2)$ 。左侧路径先 *split* 得到 $A(Y)$ 和 $B(Y)$ ，再分别做 *leaf* C2S_n ，得到 A 和 B 的系数向量，即 $(a_0, a_2, \dots)$ 与 $(a_1, a_3, \dots)$ 。右侧路径先做 *top* C2S_N ，得到 P 的完整系数向量 $(a_0, a_1, a_2, a_3, \dots)$ ，再按偶/奇下标分组。两条路径输出相同。这一例子也说明：引理不是说 *split* 保留了原始 *big slots*，而是说 *split* 后接 C2S_n 恰好得到 *top* C2S_N 后的系数子序列。

4.4 EvalMod compatibility 的证明

证明. 令 $x = (x_0, \dots, x_{N-1}) \in \text{Coeff}_N$ 为 top C2S_N 后的 coefficient-packed vector。根据 Π_k 的定义， $\Pi_k x = (S_0 x, \dots, S_{k-1} x)$ ，其中第 r 个 leaf 的第 j 个坐标为 $(S_r x)_j = x_{r+jk}$ 。

左侧路径先分组，再在每个 leaf 上做 $\text{EvalMod}_n = f^{\oplus n}$ 。因此左侧输出在第 r 个 leaf、第 j 个坐标处为

$$\begin{aligned} \left[\left(\bigoplus_{r=0}^{k-1} \text{EvalMod}_n \right) (\Pi_k x) \right]_{r,j} &= [\text{EvalMod}_n(S_r x)]_j \\ &= f((S_r x)_j) \\ &= f(x_{r+jk}). \end{aligned}$$

右侧路径先在 top vector 上做 $\text{EvalMod}_N = f^{\oplus N}$ ，得到

$$\text{EvalMod}_N(x) = (f(x_0), f(x_1), \dots, f(x_{N-1})).$$

再由 Π_k 按 residue class 分组。因此右侧输出在第 r 个 leaf、第 j 个坐标处为

$$\begin{aligned} [\Pi_k(\text{EvalMod}_N(x))]_{r,j} &= [\text{EvalMod}_N(x)]_{r+jk} \\ &= f(x_{r+jk}). \end{aligned}$$

左右两侧每个坐标都相等，因此

$$\left(\bigoplus_{r=0}^{k-1} \text{EvalMod}_n \right) \circ \Pi_k = \Pi_k \circ \text{EvalMod}_N.$$

□

这个证明没有使用 f 的线性性，只使用了同一个 scalar function f 在每个坐标上独立作用。因此即使 EvalMod 是非线性的，只要它是 coordinate-wise map，它仍然可以穿过这种 residue-class grouping。这一点正是本文能够把非线性 bootstrapping core 分解到 leaf 上的关键。

例如 11 (逐坐标函数为什么可以穿过分组)。设 $N = 4, k = 2, n = 2$ ，并令 $x = (x_0, x_1, x_2, x_3)$ 。分组后得到 (x_0, x_2) 和 (x_1, x_3) 。先分组再应用 f ，得到 $(f(x_0), f(x_2))$ 和 $(f(x_1), f(x_3))$ 。先对 x 整体逐坐标应用 f ，再分组，得到的也是同一结果。因此这里不需要 $f(a + b) = f(a) + f(b)$ 之类的性质。

4.5 S2C-Merge compatibility 的证明

证明. 任取 EvalMod 后的大环 coefficient-packed vector $y = (y_0, \dots, y_{N-1}) \in \text{Coeff}_N$ 。经过 Π_k 分组后，第 r 个 leaf 得到

$$S_r y = (y_r, y_{r+k}, \dots, y_{r+(n-1)k}).$$

将 $S_r y$ 解释为小环多项式

$$Y_r(Y) = \sum_{j=0}^{n-1} y_{r+jk} Y^j$$

的系数向量。

先看左侧路径。对每个 leaf 应用 S2C_n 后，第 r 个 leaf 得到 $\text{can}_n(Y_r)$ 。随后 Merge_k 按 coefficient/module assembly 的规则把这些 leaf 消息合成一个大环消息：

$$Y(X) = \sum_{r=0}^{k-1} X^r Y_r(X^k).$$

把 Y_r 展开，得到

$$\begin{aligned} Y(X) &= \sum_{r=0}^{k-1} X^r \left(\sum_{j=0}^{n-1} y_{r+jk} (X^k)^j \right) \\ &= \sum_{r=0}^{k-1} \sum_{j=0}^{n-1} y_{r+jk} X^{r+jk}. \end{aligned}$$

由于每个下标 $i \in \{0, \dots, N-1\}$ 都唯一写成 $i = r + jk$ ，上式正是

$$Y(X) = \sum_{i=0}^{N-1} y_i X^i.$$

因此，左侧路径最终输出的是系数向量为 y 的大环多项式的 canonical-slot 表示，即 $\text{can}_N(Y)$ 。

右侧路径 S2C_N 则直接把 coefficient-packed vector y 解释为大环多项式 $\sum_i y_i X^i$ 的系数向量，并输出其 canonical-slot 表示。因此右侧输出同样是 $\text{can}_N(Y)$ 。所以

$$\text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{S2C}_n \right) \circ \Pi_k = \text{S2C}_N.$$

□

例如 12 ($k = 2$ 时的 S2C-Merge compatibility). 设 $y = (y_0, y_1, y_2, y_3)$ 。分组得到偶分支 (y_0, y_2) 和奇分支 (y_1, y_3) , 对应小环多项式 $Y_0(Y) = y_0 + y_2Y$ 和 $Y_1(Y) = y_1 + y_3Y$ 。leaf S2C_n 后再 merge, 得到

$$Y_0(X^2) + XY_1(X^2) = y_0 + y_1X + y_2X^2 + y_3X^3,$$

这正是 top S2C_N 会解释的同一个 coefficient-packed vector。

4.6 CKKS core factorization 的证明

证明. 记 leaf core 为 $\text{BTS}_n^{\text{core}} = \text{S2C}_n \circ \text{EvalMod}_n \circ \text{C2S}_n$ 。从式 (1) 的右侧出发, 逐步使用前面三个兼容性引理:

$$\begin{aligned} & \text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} \text{S2C}_n \circ \text{EvalMod}_n \circ \text{C2S}_n \right) \circ \text{Split}_k \\ &= \text{Merge}_k \circ \left(\bigoplus_r \text{S2C}_n \right) \circ \left(\bigoplus_r \text{EvalMod}_n \right) \circ \left(\bigoplus_r \text{C2S}_n \right) \circ \text{Split}_k \\ &= \text{Merge}_k \circ \left(\bigoplus_r \text{S2C}_n \right) \circ \left(\bigoplus_r \text{EvalMod}_n \right) \circ \Pi_k \circ \text{C2S}_N \\ &= \text{Merge}_k \circ \left(\bigoplus_r \text{S2C}_n \right) \circ \Pi_k \circ \text{EvalMod}_N \circ \text{C2S}_N \\ &= \text{S2C}_N \circ \text{EvalMod}_N \circ \text{C2S}_N. \end{aligned}$$

第一步只是展开 direct sum 中的 leaf core; 第二步使用 Lemma 1; 第三步使用 Lemma 2; 第四步使用 Lemma 3。因此 CKKS bootstrapping core 的 factorization 成立。□

这个定理说明, 理想模型中的分解路径不是在 split 后恢复原始大环 slots, 而是直接利用 $\text{Split}_k + \text{leaf C2S}_n$ 获得大环 C2S_N 输出的 coefficient 子序列。由于 EvalMod 是逐坐标函数, 这些子序列可以在 leaf 上独立处理; 最后 leaf S2C_n 与 Merge_k 再把结果重组成 top S2C_N 的输出。

从密文语义看, 若 ring switching 实现了 $\text{ct}_N \models_{\Delta, N} P(X) \mapsto (\text{ct}_{n,r} \models_{\Delta, n} P_r(Y))_r$, 那么 leaf bootstrapping core 就是在这些 P_r 上分别执行同一个理想 core。理论等式保证了这些 leaf core 的消息语义可以 merge 回 top core 的消息语义。ciphertext-level 的 key-switching noise、rescale error 和 implementation scaling 不属于本节的代数等式, 而是在算法与实现章节中纳入误差分析。

4.7 General split-compatible sandwich theorem

上面的 CKKS core factorization 其实是一个更一般结构命题的实例。这个抽象定理有助于说明本文证明的真正原因: 不是 CKKS 的某个实现细节偶然成立, 而是“前向线性变换-逐坐标非线性-反向线性变换”这种 sandwich structure 与 split/merge 兼容。

定理 2 (General split-compatible sandwich theorem). 设 $F_N = L_N^{(2)} \circ \Phi_N \circ L_N^{(1)}$, 其中 $L_N^{(1)}$ 和 $L_N^{(2)}$ 是线性映射, $\Phi_N = \phi^{\oplus N}$ 是逐坐标映射。设 leaf 电路为 $F_n = L_n^{(2)} \circ \Phi_n \circ L_n^{(1)}$, 其中

$\Phi_n = \phi^{\oplus n}$ 。若存在 *layout map* Π_k ，使得

$$\left(\bigoplus_{r=0}^{k-1} L_n^{(1)} \right) \circ \text{Split}_k = \Pi_k \circ L_N^{(1)}$$

且

$$\text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} L_n^{(2)} \right) \circ \Pi_k = L_N^{(2)},$$

则

$$F_N = \text{Merge}_k \circ \left(\bigoplus_{r=0}^{k-1} F_n \right) \circ \text{Split}_k.$$

证明. 从右侧出发，先展开 leaf 电路：

$$\begin{aligned} & \text{Merge}_k \circ \left(\bigoplus_r F_n \right) \circ \text{Split}_k \\ &= \text{Merge}_k \circ \left(\bigoplus_r L_n^{(2)} \right) \circ \left(\bigoplus_r \Phi_n \right) \circ \left(\bigoplus_r L_n^{(1)} \right) \circ \text{Split}_k. \end{aligned}$$

由第一个兼容条件， $(\bigoplus_r L_n^{(1)}) \circ \text{Split}_k = \Pi_k \circ L_N^{(1)}$ 。代入得

$$\text{Merge}_k \circ \left(\bigoplus_r L_n^{(2)} \right) \circ \left(\bigoplus_r \Phi_n \right) \circ \Pi_k \circ L_N^{(1)}.$$

接下来，因为 $\Phi_N = \phi^{\oplus N}$ 、 $\Phi_n = \phi^{\oplus n}$ ，而 Π_k 只是把坐标按 leaf layout 分组，所以逐坐标映射与该分组交换：

$$\left(\bigoplus_r \Phi_n \right) \circ \Pi_k = \Pi_k \circ \Phi_N.$$

为了看清这一点，取任意 $x = (x_0, \dots, x_{N-1})$ ，第 r 个 leaf 的第 j 个坐标在左侧为 $\phi(x_{r+jk})$ ，在右侧也为 $\phi(x_{r+jk})$ 。因此上式成立。

于是整体变为

$$\text{Merge}_k \circ \left(\bigoplus_r L_n^{(2)} \right) \circ \Pi_k \circ \Phi_N \circ L_N^{(1)}.$$

再由第二个兼容条件， $\text{Merge}_k \circ (\bigoplus_r L_n^{(2)}) \circ \Pi_k = L_N^{(2)}$ 。因此得到

$$L_N^{(2)} \circ \Phi_N \circ L_N^{(1)} = F_N.$$

定理得证。 □

例如 13 (CKKS core 是 sandwich theorem 的实例). 在 *CKKS bootstrapping core* 中，取 $L^{(1)} = \text{C2S}$ 、 $\Phi = \text{EvalMod}$ 、 $L^{(2)} = \text{S2C}$ 。Lemma 1 给出第一个线性兼容条件，Lemma 3 给出第二个线性兼容条件，Lemma 2 正是逐坐标中间层与 *layout grouping* 的交换性。因此 Theorem 1 是 Theorem 2 的直接实例。

4.8 本节小结

本节证明了三件事。第一， Split_k 后接 leaf C2S_n 的效果等价于 top C2S_N 后的 coefficient residue-class 分组。第二， EvalMod 虽然是非线性的，但它是逐坐标函数，因此可以在分组后的 leaf vectors 上独立执行。第三，leaf S2C_n 后接 Merge_k 可以重构 top S2C_N 的输出。三者合起来给出 CKKS bootstrapping core 的 subring factorization。

这也解释了为什么本文不需要在 split 后先做 slot-preserving recovery。本文分解的对象不是原始 big-ring canonical slots，而是 bootstrapping core 中 C2S 后的 coefficient-packed coordinates。下一节将在该代数结论的基础上讨论 ciphertext-level algorithm、ring switching noise、复杂度和参数选择。

[待完成：实现章节需要补充：Lattigo 的 half-slot convention、real/imag branches、DFT factorization、basis ordering 与本节 normalized layout 的对应关系。本节不展开 implementation scaling。]

5 算法、正确性与参数分析

5.1 Factored bootstrapping algorithm

Ciphertext-level algorithm 使用 ring switching 在真实密钥下实现 algebraic split。

Algorithm 1 Subring-factored CKKS bootstrapping

Require: 大环 ciphertext ct_N , ring-switching keys, leaf bootstrapping keys。

- 1: $(ct_{n,0}, \dots, ct_{n,k-1}) \leftarrow \text{RSDown}_{N \rightarrow n}(ct_N)$ 。
 - 2: **for** $r = 0, \dots, k-1$ 并行执行 **do**
 - 3: $ct'_{n,r} \leftarrow \text{BTS}_n(ct_{n,r})$ 。
 - 4: **end for**
 - 5: $ct'_N \leftarrow \text{RSUp}_{n \rightarrow N}(ct'_{n,0}, \dots, ct'_{n,k-1})$ 。
 - 6: **return** ct'_N 。
-

代数定理说明 leaf-wise core 为什么正确。算法还要求 RSDown 输出 leaf secret 下有效的 leaf ciphertexts，并要求 RSUp 能回到目标 top secret。

5.2 正确性与噪声

假设 ct_N 解密为 $m + e$ 。ring switching down 之后，我们要求

$$\text{Dec}_{s_n}(ct_{n,r}) = m_r + e_r,$$

其中 m_r 是 m 的第 r 个 coefficient component，且 $\|e_r\|$ 在 leaf bootstrapping 的输入容忍范围内。如果每个 leaf bootstrapping 成功，则 $ct'_{n,r}$ 解密为 $m_r + e'_r$ ，其中 e'_r 是刷新后的误差。merge 与 switch up 后得到一个解密为 $m + e_{\text{out}}$ 的 ciphertext。

一个保守的 failure decomposition 是

$$\Pr[\text{fail}] \leq \delta_{\text{RSDown}} + k\delta_{\text{BTS},n} + \delta_{\text{RSUp}}.$$

噪声来源包括输入 CKKS error、ring-switching key-switching error、leaf linear-transform error、EvalMod approximation error、rescale/RNS rounding error 和 final switch-up error。

[待完成：给出更紧的噪声界，或者明确标注这一部分只是 conservative framework。]

5.3 复杂度与 overhead

令 $T_{\text{shared}}(N)$ 表示仍保留在 top-level 的 preprocessing，令 $T_{\text{core}}(N)$ 表示 C2S_N 、 EvalMod_N 和 S2C_N 的成本。Direct cost 建模为

$$T_{\text{dir}}(N) = T_{\text{shared}}(N) + T_{\text{core}}(N).$$

Factored cost 建模为

$$T_{\text{fac}}(N, k) = T_{\text{shared}}(N) + T_{\text{RSDown}}(N, k) + T_{\text{leaf}}^{\parallel}(n, k) + T_{\text{RSUp}}(N, k).$$

如果有 w 个 workers，parallel efficiency 为 η_{par} ，则一阶近似为 $T_{\text{leaf}}^{\parallel} \approx kT_{\text{core}}(n)/(\min(k, w)\eta_{\text{par}})$ 。预期加速来自用若干个小 core 替换一个大 core；额外成本来自 ring switching、extra switching keys、leaf bootstrapping keys、synchronization 和 memory。

[待完成：补充 measured per-stage timings、key generation time、key sizes 和 peak memory。]

5.4 安全性与参数选择

安全性必须对所有出现公开 RLWE samples 的 rings 进行估计。特别地，leaf security 不会自动由 top-ring security 继承。如果 leaf ciphertexts 和 leaf evaluation keys 生活在 $R_{q,n}$ 中，则相关估计需要使用 leaf dimension n 以及这些 keys 使用的最大公开 modulus，通常可保守近似为 QP 。

这带来一个张力：更小的 n 能加速 leaf computation，但 leaf modulus 必须足够小才能满足安全性。因此，一个 security-aware parameter table 至少应报告：top N 与 QP 、leaf n 与 $Q_{\text{leaf}}P_{\text{leaf}}$ 、ring-switching key moduli、bootstrapping key moduli，以及所有估计中的最小安全位数。

Tier	$\log N$	k	$\log n$	定位
Prototype	16	4	14	机制与正确性验证
Prototype+	16	2	15	正确性与 timing
Security-aware	17	2	16	当前主实验
Heavy target	18	4	16	后续高吞吐目标

表 1: 计划中的参数层级。

[待完成：对 top keys、leaf keys、RSDown keys、RSUp keys 和 bootstrapping keys 运行 lattice estimator。只有完成表格后，才能将 security-aware language 替换为具体 bit-security。]

5.5 安全证明

这里得加一个安全性证明。

6 实现与实验评估

6.1 Implementation setup

Prototype 基于 Lattigo 实现。实现使用 top parameter set 处理输入和输出 ciphertext，使用 leaf parameter sets 执行 subring bootstrappings，使用 ring-switching keys 实现 RSDown 和 RSUp，并使用 leaf evaluators pool 执行并行计算。需要检查 top 与 leaf bases 的兼容性，包括 exact Q/P basis、NTT-prime compatibility、scale conventions 和 layout ordering。

[待完成：补充 Lattigo version、hardware、workers 数量、exact parameter literals 和 key-generation settings。]

6.2 Layout and normalization diagnostics

理想代数定理并不规定 Lattigo 的 implementation scaling。实践中我们观察到

$$\text{BTS}_{N,k}^{\text{fac,impl}} \approx \gamma_{\text{impl}} \text{BTS}_N^{\text{impl}}.$$

其中 γ_{impl} 是 C2S 与 S2C 的 effective scaling choices 带来的 implementation convention，不是 factorization theorem 的数学修正。有些参数组 RAW 形式直接对齐，另一些参数组需要显式除一个小因子，例如 k 。

因此，implementation 应记录 top C2S/S2C scaling、leaf C2S/S2C scaling、output scale、level、RAW error、normalized error 和 fitted γ 。

[待完成：插入 $\log N = 17, k = 2$ 和 $\log N = 16, k = 2$ 实验的 normalization diagnostic table。]

6.3 Correctness experiments

Core correctness experiments 应包括 Split/Merge roundtrip、C2S alignment、after-EvalMod alignment、final direct-vs-factored error、output level 和 normalization behavior。当前计划中的主实验是

$$\log N = 17, \quad k = 2, \quad \log n = 16.$$

已有运行显示，该设置可以获得接近的 direct-vs-factored alignment 和显著 speedup。

[待完成：插入最终表格，包括 C2S alignment、after-EvalMod alignment、final error、output level，以及多次 trials 下的 failure rate。]

6.4 Performance and overhead

Performance table 应报告 direct full bootstrapping time、factored full bootstrapping time、speedup、per-stage timings、worker count、key size 和 memory。更有用的表格应分别列出 RSDown、leaf C2S、leaf EvalMod、leaf S2C 和 RSUp。

[待完成：补 key size 和 peak memory。只有 timing 还不足以支撑完整 systems claim。]

6.5 RNS-aware reduced- Q investigation

一个核心开放问题是：switch 到 subrings 之后，leaf bootstrapping 是否可以使用更小的 modulus chain Q_{leaf} 或更小的公开 modulus $Q_{\text{leaf}}P_{\text{leaf}}$ 。这很重要，因为 leaf security 要在 dimension n 下估计。（待完成）

在 coefficient/RNS 层面，basis extension、modulus-chain restriction 和 coefficient-wise RNS operations 与 coefficient split 交换。然而，Lattigo 实验显示，现有 bootstrapping preprocessing，例如 Evaluator.ModUp 或组合的 ScaleDown + ModUp block，不能简单地移到 split 之后再在 leaves 上独立调用。因此，正确的理论目标不是与 direct path 的中间态相等。Reduced- Q factored bootstrapping 应该被表述为一个独立的 leaf bootstrapping design problem：low-modulus RSDown 必须输出有效 leaf ciphertexts，并且其噪声需要落在由 Q_{leaf} 参数化的 leaf bootstrapping 输入容忍范围内。

[待完成：Reduced- Q leaf bootstrapping 尚未完成。当前证据排除了最简单的 Lattigo preprocessing 重排，但没有排除 custom low-level preprocessing 或新的 leaf parameter set。]

7 讨论与限制

当前草稿已经建立 CKKS bootstrapping core 的代数分解、split 的 coefficient/slot interpretation，以及通过 subring ring switching 实现 ciphertext-level algorithmic path 的结构。它也给出了实现诊断和参数分析的框架。但仍有若干部分尚未完成。

[待完成：Reduced- Q leaf bootstrapping 仍然是开放问题。已有实验表明 Lattigo preprocessing 不能直接下放到 leaves；需要 custom RNS/preprocessing layer 或新的 leaf bootstrapping parameter design。]

[待完成：完整安全估计缺失。最终论文必须对 top、leaf、RSDown、RSUp 和 bootstrapping keys 跑 lattice estimator，并报告最小 bit security。]

[待完成：噪声分析目前是 conservative framework。需要加入更紧的 bound，或者至少加入经过验证的 empirical precision model。]

[待完成：Key size、peak memory 和 per-stage timing 尚未完整。这些信息对于评估 ring switching 和 leaf keys 的 overhead 是必要的。]

[待完成：更大 k 的 security-aware 参数实验尚未完成。除非 leaf security 得到确认，否则 $k = 4$ prototype 不应作为安全主实例呈现。]

最后，preliminaries 中讨论的 slot-lifting 或 slot-preserving transform 不是主算法的一部分。它是有用的比较对象，也有助于理解其他语义；但本文提出的 bootstrapping factorization 工作在 coefficient/module layout 中。

8 结论

本文草稿提出了一条 subring-factored CKKS bootstrapping 路线。主要结果是传统 C2S $\rightarrow$ EvalMod $\rightarrow$ S2C core 与 coefficient/module decomposition 兼容：一个大环 core 可以被替换为

独立的 leaf cores 加一次 merge。这不同于 ring packing、subring-secret acceleration 和 PaCo 这类新 bootstrapping 设计。本文保留传统 CKKS bootstrapping core，但改变其执行粒度。下一步需要完成严格证明、补齐实现表格、运行完整安全估计，并判断 reduced- Q leaf bootstrapping 是否能够做到正确且安全。

参考文献

- [1] C. Gentry, S. Halevi, and N. P. Smart. Ring switching in BGV-style homomorphic encryption. Preliminary version, 2012.
- [2] Y. Bae, J. H. Cheon, J. Kim, J. H. Park, and D. Stehle. HERMES: Efficient ring packing using MLWE ciphertexts and application to transciphering. 2024.
- [3] S. Ma, T. Huang, A. Wang, and X. Wang. Practical dense-key bootstrapping with subring secret encapsulation. 2025.
- [4] J.-S. Coron and T. Seure. PaCo: Bootstrapping for CKKS via partial CoeffToSlot. 2025.
- [5] J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic encryption for arithmetic of approximate numbers. ASIACRYPT 2017.